



SQL – DDL & DQL

Agenda

- ▶ **SQL**
- ▶ **Data Definition Language**
- ▶ **Data Manipulation Language**
- ▶ **Examples & Exercises**



SQL

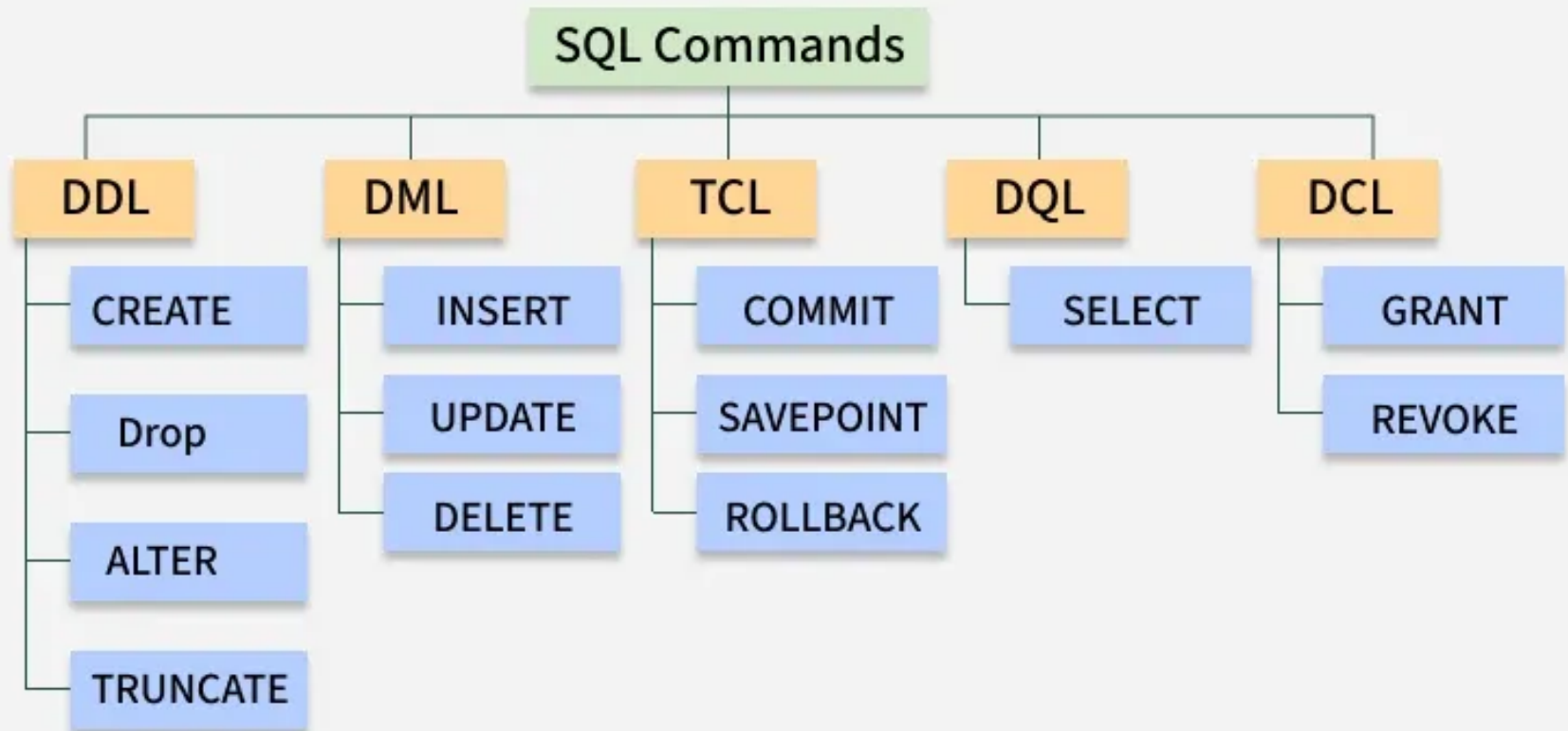
SQL

- ▶ **IBM developed the original version of SQL, originally called Sequel, as part of the System R project in the early 1970s.**
- ▶ **The Sequel language has evolved since then, and its name has changed to SQL (Structured Query Language).**

SQL

- ▶ **Several Versions have appeared such as SQL-86; followed by SQL-92 standard, SQL:1999, SQL:2003, SQL:2006, SQL:2008, SQL:2011.**
- ▶ **The current international standard is ISO/IEC 9075:2023 (often called SQL:2023..**

SQL Commands





SQL - Data Definition Language

SQL - DDL

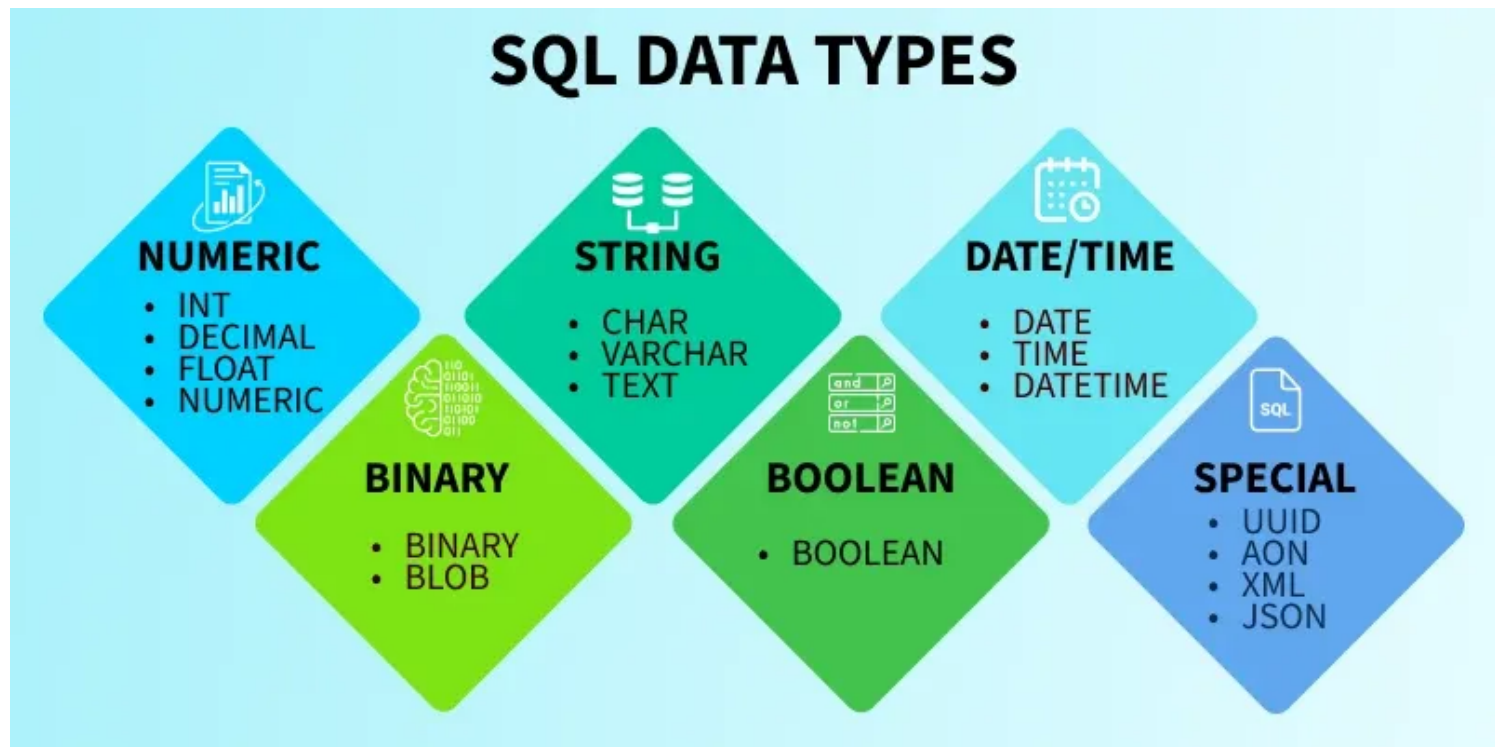
- ▶ It is a subset of SQL commands used to define, modify, and manage the structure of database objects such as tables, schemas, indexes, and views.
- ▶ It simply deals with descriptions of the database schema and is used to create and modify the structure of database objects in the database

SQL - DDL

- ▶ **It Includes Commands:**
 - ▶ **Create**
 - ▶ **Alter**
 - ▶ **Drop**
 - ▶ **Truncate**

SQL – Domain Types

- ▶ SQL Standard supports variety of built-in data types:



Domain Types - Integer

Data Type	Description	Range
BIGINT	Large integer numbers	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
INT	Standard integer values	-2,147,483,648 to 2,147,483,647
SMALLINT	Small integers	-32,768 to 32,767
TINYINT	Very small integers	0 to 255
DECIMAL	Exact fixed-point numbers (e.g., for financial values)	$-10^{38} + 1$ to $10^{38} - 1$

Domain Types - Character

Data Type	Description
Char	The maximum length of 8000 characters. (Fixed-Length non-Unicode Characters)
Varchar	The maximum length of 8000 characters. (Variable-Length non-Unicode Characters)
Varchar(max)	The maximum length of $2^{31} - 1$ characters(SQL Server 2005 only). (Variable Length non-Unicode data)
Text	The maximum length of 2,127,483,647 characters(Variable Length non-Unicode data)

Domain Types – Date and Time

Data Type	Description	Storage Size
DATE	stores the data of date (year, month, day)	3 Bytes
TIME	stores the data of time (hour, minute,second)	3 Bytes
DATETIME	store both the data and time (year, month, day, hour, minute, second)	8 Bytes

Integrity Constraints

- ▶ **Primary Keys:** Specified as primary key (A1, A2, ..., An)
- ▶ **Foreign Keys:** Specified as foreign key (A1, A2, ..., An) references r
 - ▶ Foreign key used must be the primary key of referenced table, i.e. 'r'
- ▶ **Non-null values:** Restricting the attribute to never have null values
 - ▶ Specified as: not null
- ▶ If any of the integrity constraints are violated, SQL will not perform the update

SQL – DDL - CREATE

- ▶ **Relations or Tables need to be defined first before any operations can be performed.**
- ▶ **The CREATE TABLE command in SQL defines how your data will be stored, including the table name, column names, data types, and rules (constraints) such as NOT NULL, PRIMARY KEY etc...**

SQL – DDL - CREATE

Command	Description	Syntax
<u>CREATE</u>	Create database or its objects (table, index, function, views, store procedure and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);

```
CREATE DATABASE UniversityDB;
```


SQL – DDL - CREATE

► Example: Creating a Student Relation

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Age INT CHECK (Age >= 18),  
    Email VARCHAR(100) UNIQUE  
);
```

SQL – DDL - CREATE

- Example: Creating a Student Relation with PK defined at Table-level

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(50),  
    Age INT,  
    CONSTRAINT pk_student PRIMARY KEY (StudentID)  
);
```

SQL – DDL - CREATE

- ▶ Example: Creating a Enrollment Relation with FK defined inline

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT REFERENCES Student(StudentID), -- FK inline  
    CourseID INT REFERENCES Course(CourseID),    -- FK inline  
    Grade CHAR(1)  
);
```

SQL – DDL - CREATE

- ▶ Example: Creating a Enrollment Relation with FK defined at Table-level

```
CREATE TABLE Enrollment (  
    EnrollmentID INT PRIMARY KEY,  
    StudentID INT,  
    CourseID INT,  
    Grade CHAR(1),  
    CONSTRAINT fk_student FOREIGN KEY (StudentID) REFERENCES Student(StudentID),  
    CONSTRAINT fk_course FOREIGN KEY (CourseID) REFERENCES Course(CourseID)  
);
```

SQL – DDL - CREATE

- ▶ Example: Creating a Enrollment Relation with composite PK

```
CREATE TABLE Enrollment (  
    StudentID INT,  
    CourseID INT,  
    EnrollmentDate DATE,  
    Grade CHAR(1),  
    CONSTRAINT pk_enrollment PRIMARY KEY (StudentID, CourseID) -- Composite PK  
);
```


SQL – DDL - CREATE

- Example: Creating a Payment Relation with composite FK

```
CREATE TABLE Payment (  
    StudentID INT,  
    CourseID INT,  
    Amount DECIMAL(10,2),  
    CONSTRAINT fk_enrollment FOREIGN KEY (StudentID, CourseID)  
        REFERENCES Enrollment(StudentID, CourseID)  
);
```

SQL – DDL - ALTER

- ▶ **SQL ALTER TABLE** statement modify the structure of an existing table in a database.
- ▶ Whether adding new columns, modifying existing ones, deleting columns or renaming them.
- ▶ **ALTER TABLE** statement enables you to make changes without losing data stored in the table.

SQL – DDL - ALTER

- ▶ **Syntax:**

ALTER TABLE *table_name* [ADD | DROP | MODIFY]
column_name *datatype*

- ▶ **Add – Used to add new column**
- ▶ **Drop – Used to remove existing column**
- ▶ **Modify – Used to change datatype or definition of existing column**

SQL – DDL - ALTER

- ▶ **Example:**
 - ▶ Considering the following Students Relation
 - ▶ Students(ID, Name, Age, Dep_Name)
 - ▶ Adding an email column

```
ALTER TABLE Students  
ADD Email varchar(255);
```

SQL – DDL - ALTER

- ▶ **Adding multiple columns:**
 - ▶ **Considering the following Students Relation**
 - ▶ **Student(Roll_no, Name)**
 - ▶ **Adding Age and course columns**

**ALTER TABLE Student ADD
(AGE number(3), Course varchar(40));**

SQL – DDL - ALTER

ROLL_NO	NAME	AGE	COURSE
1	Emma	null	null
2	Travis	null	null
3	Jennifer	null	null
4	Roxanne	null	null

SQL – DDL - ALTER

► DROP

- DROP clause allows you to remove a column from a table. Be cautious when using this command as it will permanently remove the column and its data.

ALTER TABLE Student DROP COLUMN
column_name

► *Example*

- **ALTER TABLE Student DROP COLUMN Email;**

SQL – DDL - ALTER

► MODIFY / ALTER

- The MODIFY (or ALTER COLUMN in some databases like SQL Server) clause is used to modify the definition of an existing column, such as changing its data type or size.

ALTER TABLE Student ALTER COLUMN
column_name datatype

ALTER TABLE Student ALTER COLUMN Age
Varchar(3);

SQL – DDL – DROP

- ▶ ***The SQL DROP command is used to permanently remove an object from a database, such as a table, database, index or view. When a table is dropped, both its data and structure are permanently removed, and the action cannot be undone without a backup.***
 - ▶ ***Permanent Removal: Deletes object and data permanently.***
 - ▶ ***Deletes Dependencies: Removes related constraints and indexes.***
 - ▶ ***Schema Deleted: Removes table structure and schema.***
 - ▶ ***No Integrity Check: Skips constraints and triggers.***

SQL – DDL – DROP

- ▶ ***Syntax***

- ▶ ***DROP TABLE table_name;***

- ▶ ***Example***

- ▶ ***DROP TABLE student_details;***

SQL – DDL – DROP

- ▶ *The DROP and TRUNCATE commands in SQL are used to remove data from a table, but they work differently. This difference is crucial for effective database management, especially with large datasets.*
- ▶ *The TRUNCATE command is a Data Definition Language (DDL) action that removes all rows from a table but preserves the structure of the table for future use.*

SQL – DDL – DROP

- ▶ ***Syntax***

- ▶ ***TRUNCATE TABLE table_name;***

- ▶ ***Example***

- ▶ ***TRUNCATE TABLE student_details;***

Activity

Part 1: CREATE

1. Create a database named **UniversityDB**.
2. Inside it, create a table **Department** with the following columns:
 - **DeptID** (INT, Primary Key)
 - **DeptName** (VARCHAR(50), Not Null, Unique)
 - **Location** (VARCHAR(50))
3. Create a table **Student** with:
 - **StudentID** (INT, Primary Key)
 - **Name** (VARCHAR(50))
 - **Age** (INT, must be ≥ 18)
 - **DeptID** (INT, Foreign Key referencing Department)

Activity

Part 2: ALTER

1. Add a new column Email (VARCHAR(100)) to **Student**.

2. Modify the Age column so that it cannot be NULL.

3. Drop the Location column from
Department.

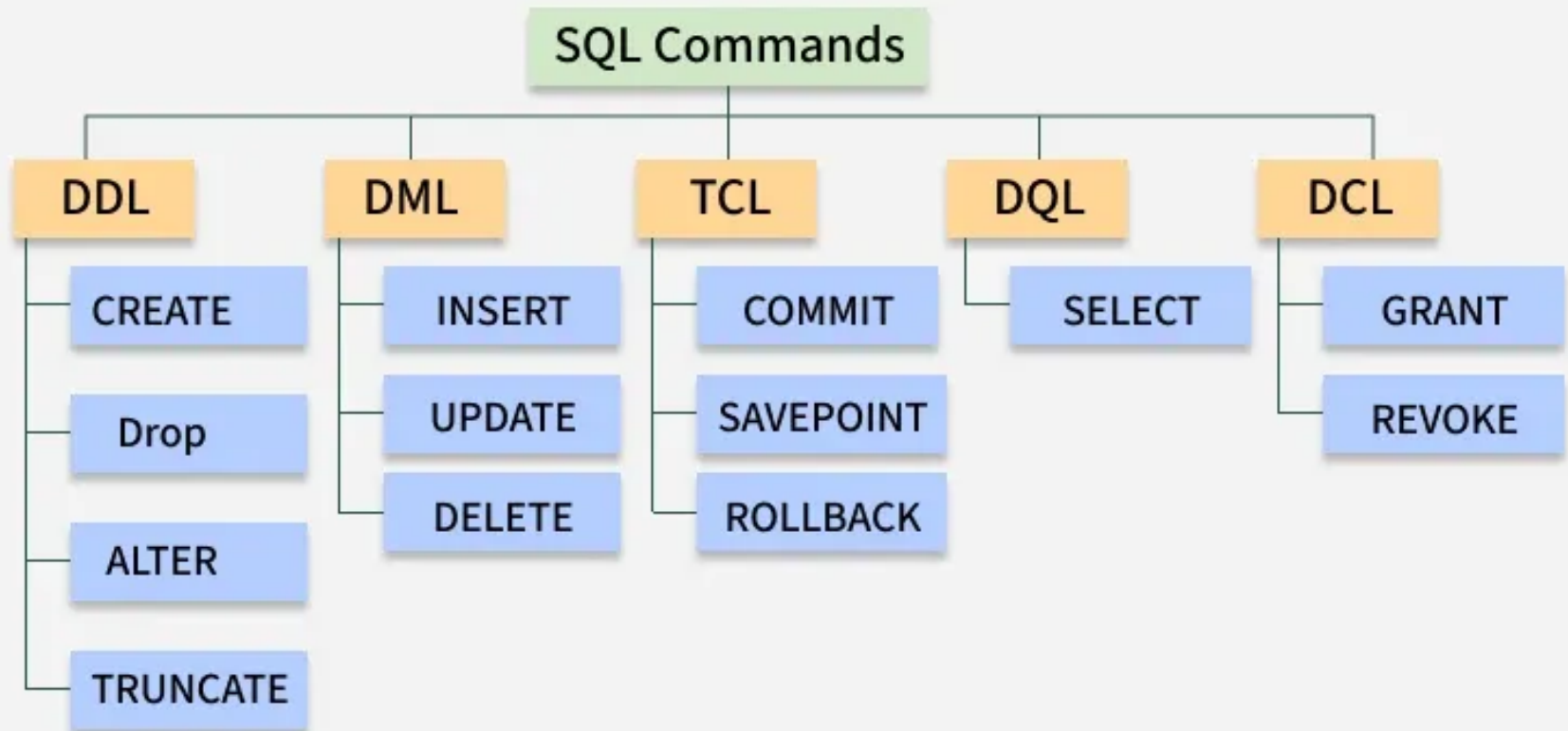
Activity

- ▶ **Part 3: DROP**
- ▶ Drop the **Learner** table.
- ▶ Drop the **Department** table.



SQL - Data Query Language

SQL Commands



SQL - DQL

- ▶ **DQL is used to fetch data from the database. The main command is SELECT, which retrieves records based on the query.**
- ▶ **The output is returned as a result set (a temporary table) that can be viewed or used in applications.**

SQL – DQL – Select

- ▶ **SQL SELECT is used to retrieve data from one or more tables, either all records or specific results based on conditions.**
- ▶ **It returns output in a tabular format of rows and columns.**

SQL – DQL – Select

- ▶ **Extracts data from tables.**
- ▶ **Targets specific or all columns (*).**
- ▶ **Supports filtering, sorting, grouping, and joins.**
- ▶ **Results are stored in a result set.**

SQL – DQL – Select

- ▶ **Select All Columns**

- ▶ **Syntax**

Select * from TABLE_NAME;

- ▶ **Example**

Select * from Customer

SQL – DQL – Select

CustomerID	CustomerName	Country	City	Phone
C001	Ali Khan	Pakistan	Karachi	0300-1234567
C003	Imran Siddiqui	Pakistan	Lahore	0321-9876543
C005	Fatima Noor	Pakistan	Islamabad	0333-5556667

SQL – DQL – Select

- ▶ **Select Specific Columns from the table**

- ▶ **Syntax**

**Select COLUMN_NAME1, COLUMN_NAME2,...
from TABLE_NAME;**

- ▶ **Example**

**Select CustomerName, City, Phone from
Customer;**

SQL – DQL – Select

CustomerName	City	Phone
Ali Khan	Karachi	0300-1234567
Sara Ahmed	New York	+1-555-6789
Imran Siddiqui	Lahore	0321-9876543
Fatima Noor	Islamabad	0333-5556667

SQL – DQL – Select

Example

Select CustomerID, CustomerName from Customer;

CustomerID	CustomerName
C001	Ali Khan
C002	Sara Ahmed
C003	Imran Siddiqui
C005	Fatima Noor

SQL – DQL – Select

▶ SELECT Statement with WHERE Clause

▶ Syntax

```
Select  COLUMN_NAME1, COLUMN_NAME2,...  
from TABLE_NAME  
WHERE CONDITION;
```

CustomerName

Ali Khan

▶ Example

```
Select CustomerName, from Customer  
Where City = 'Karachi'
```

SQL – DQL – Select - Where

- ▶ **The WHERE clause filters rows based on a condition.**
- ▶ **You can use operators like:**
 - ▶ **= , <> (not equal), >, <, >=, <=**
 - ▶ **AND, OR, NOT**
 - ▶ **IN (...), BETWEEN ... AND ..., LIKE 'pattern%'**

Example – Customer Table

CustomerID	CustomerName	City	Country	Phone
C001	Ali Khan	Karachi	Pakistan	0300123456
C002	Sara Ahmed	London	UK	0201234567
C003	Imran Siddiqui	Lahore	Pakistan	0300987654
C004	Fatima Noor	Islamabad	Pakistan	NULL

SQL – DQL – Select - Where

► Basic Equality

```
SELECT CustomerName, City  
FROM CUSTOMER  
WHERE City = 'Karachi';
```

CustomerName

Ali Khan

SQL – DQL – Select - Where

► Not Equal

```
SELECT CustomerName, City  
FROM CUSTOMER  
WHERE City <> 'Karachi';
```

CustomerName	City
Sara Ahmed	London
Imran Siddiqui	Lahore
Fatima Noor	Islamabad

SQL – DQL – Select - Where

► Greater Than

```
SELECT CustomerName, Phone  
FROM CUSTOMER  
WHERE CustomerID > 'C002';
```

CustomerName	Phone
Imran Siddiqui	0300987654
Fatima Noor	NULL

SQL – DQL – Select - Where

► AND Operator

SELECT CustomerName, City

FROM CUSTOMER

WHERE City = 'Karachi' AND Country = 'Pakistan';

CustomerName	City
Ali Khan	Karachi

SQL – DQL – Select - Where

► OR Operator

```
SELECT CustomerName, City  
FROM CUSTOMER  
WHERE City = 'Karachi' OR City = 'Lahore';
```

CustomerName	City
Ali Khan	Karachi
Imran Siddiqui	Lahore

SQL – DQL – Select - Where

► IN Operator

```
SELECT CustomerName, City  
FROM CUSTOMER  
WHERE City IN ('Karachi', 'Lahore', 'Islamabad');
```

CustomerName	City
Ali Khan	Karachi
Imran Siddiqui	Lahore
Fatima Noor	Islamabad

SQL – DQL – Select - Where

► NOT IN Operator

```
SELECT CustomerName, City  
FROM CUSTOMER  
WHERE City NOT IN ('Karachi', 'Lahore');
```

CustomerName	City
Sara Ahmed	London
Fatima Noor	Islamabad

SQL – DQL – Select - Where

► BETWEEN Operator

```
SELECT CustomerName, CustomerID  
FROM CUSTOMER  
WHERE CustomerID BETWEEN 'C002' AND 'C004';
```

CustomerName	CustomerID
Sara Ahmed	C002
Imran Siddiqui	C003
Fatima Noor	C004

SQL – DQL – Select - Where

▶ LIKE Operator – Pattern Matching

- ▶ Starts with A

```
SELECT CustomerName  
FROM CUSTOMER  
WHERE CustomerName LIKE 'A%';
```

CustomerName

Ali Khan

SQL – DQL – Select - Where

▶ LIKE Operator – Pattern Matching

- ▶ Ends with d

```
SELECT CustomerName  
FROM CUSTOMER  
WHERE CustomerName LIKE '%d';
```

CustomerName

Sara Ahmad

SQL – DQL – Select - Where

▶ LIKE Operator – Pattern Matching

- ▶ Contains 'ra'

```
SELECT CustomerName  
FROM CUSTOMER  
WHERE CustomerName LIKE '%ra%';
```

CustomerName
Sara Ahmed
Imran Siddiqui

SQL – DQL – Select - Where

► IS Null

```
SELECT CustomerName, Phone  
FROM CUSTOMER  
WHERE Phone IS NULL;
```

CustomerName	Phone
Fatima Noor	NULL

SQL – DQL – Select - Where

► IS NOT Null

```
SELECT CustomerName, Phone  
FROM CUSTOMER  
WHERE Phone IS NOT NULL;
```

CustomerName	Phone
Ali Khan	0300123456
Sara Ahmed	0201234567
Imran Siddiqui	0300987654



THANK YOU